

Long RL training on Kaggle

Printed from `docs/KAGGLE.md`. The complete handover, with every other document, is `docs/HANDOVER.pdf`.

Long RL training on Kaggle

DOCS/KAGGLE.MD

Long RL training on Kaggle — the complete guide

Status (27 September 2026): everything is ready, nothing has been run on Kaggle yet. The kit has only been tested on this laptop with tiny settings (a few hundred training steps on the CPU, 16 courses), so the first real Kaggle run may still show a problem that only appears there (paths, GPU memory, time). This file explains, step by step: which files to take, how to start the training on Kaggle, what gets trained, what problem it solves, how to change things, where to paste every file when training is finished, and what you get at the end. Section 12 has the whole procedure on one page.

In short. One command on the laptop builds a zip file. Upload it to Kaggle as a dataset, import our notebook, switch the GPU on and press "Save & Run All". About 11 hours later Kaggle gives you `results.zip`, which holds the newly trained policy (`best_policy.npz`) and its test results. If the new results are better than the old ones, put `best_policy.npz` in place of `models/avoid_policy.npz`: the app and the drones then use the new policy on their own. Details in section 12 and below.

1. What gets trained (in simple words)

Only one thing: the **obstacle-avoidance brain of the followers** — a small neural network.

- **Inputs (47 numbers, 10 times a second):** where the formation wants the drone to go (desired velocity), the drone's own velocity, how far it is from its place in the V, 24 distance readings around the drone (like a 2-D lidar, up to 25 m), the closest obstacle point, and the three nearest drones. Everything is turned into the formation's own frame (forward/left), so the same network works whichever way the route points (`src/swarm_agent/avoidance.py`).
- **Output (2 numbers):** a sideways/forward velocity correction (up to 8 m/s per axis) added to the formation command. The hard safety layer (drone-to-drone repulsion, geofence) and — in the "RL + brake" option — a stopping-distance brake stay on top of it.
- **Network:** 47 inputs → 128 → 128 (tanh) → 2 outputs. After training it is saved as plain numpy weights (`.npz`, about 180 KB), so the drone needs no machine-learning library.
- **How it learns:** PPO (a standard reinforcement-learning method). One shared network drives all nine followers. On the GPU, 1024 missions run at the same time (`src/swarm_tools/torch_sim.py`, checked step by step against the laptop simulator by `tests/test_torch_sim.py`).
- **What it is rewarded for (per follower, every 0.1 s):** staying close to its slot in the V, getting closer to it, small corrections; it is penalised for coming within 4 m of an obstacle, for coming closer than 5 m to another drone, and heavily (−10) for a crash. The exact formula is in `src/swarm_tools/obstacle_sim.py` (search for `# rewards for the followers`).
- **Training courses:** random routes of 300–600 m through random buildings and tree clusters (0–2 buildings and 0–1 tree clusters per hectare), 8,000 different courses.

What is **not** trained: the leader election, the formation law, the leader's route planner (A*), the autopilot. Those are rules and stay exactly as they are (see `docs/DECISIONS.md`, 14 and 17).

2. What problem it solves, and what it will not

The laptop training (6 million steps, 38 minutes on the CPU) produced the current policy. On 90 test courses it never saw (docs/RESULTS.md):

Method	Few obstacles	Medium	Dense
Classical (tuned)	15/30	7/30	3/30
RL + brake (laptop policy)	26/30	18/30	5/30

Two open problems:

1. **The training was still improving when it stopped** (reports/rl_training.png). Dense clutter is unsolved (5 of 30). Longer training — about 10 hours on a GPU per session instead of 38 minutes on the CPU — should reduce crashes, most of all at medium and high density. How much is unknown until it is measured.
2. **One training run proves little**. Training again with seeds 2 and 3 shows whether the result repeats; that is needed before anyone presents the result as reliable.

It will **not** fix: the simplified physics, the 2-D obstacles (no flying over), the perfect sensing assumed in simulation, or the radio-split separation weak spot (docs/KNOWN_ISSUES.md).

3. What you need

- A Kaggle account (free). Kaggle may ask you to verify a phone number before GPUs can be switched on.
- GPU time: every run uses one session of up to 12 hours (Kaggle's limit for GPU notebooks, from kaggle.com/docs/notebooks). Your weekly GPU quota is shown in your Kaggle account settings; plan one session per seed, so three sessions for seeds 1–3.
- On the laptop: this repository. Nothing has to be installed on Kaggle: PyTorch is already in Kaggle's image and the notebook installs nothing, so it runs with Internet off.

4. Which files to take

Only **two files** go to Kaggle:

File on the laptop	What it is	How it gets to Kaggle
kaggle/swarm-rl-code.zip	all the code (src/ , scripts/ , config/ , cached map data data/osm/ , pyproject.toml), about 0.5 MB	uploaded as a private Dataset named <code>swarm-rl-code</code>
kaggle/train_swarm_rl.ipynb	the notebook that runs everything	imported as a notebook

Make the zip fresh every time (it is not in git), from the repository root:

```
python3 scripts/make_kaggle_bundle.py
```

It prints wrote .../kaggle/swarm-rl-code.zip (N files, 0.5 MB; no *.local.* files, no map keys). Your map keys (config/map_keys.local.yaml) are never packed: the script skips *.local.* files and refuses to build if any packed file contains one of your key values. (Fixed on 27 September 2026 — an

earlier zip had included the keys file; that zip was never uploaded.)

5. Starting the training on kaggle.com, click by click

1. **Upload the code.** kaggle.com → **Create** → **New Dataset** → drag `kaggle/swarm-rl-code.zip` in → title `swarm-rl-code` → keep it **Private** → **Create**. (Kaggle may unpack the zip; the notebook handles both cases.)
2. **Import the notebook.** **Create** → **New Notebook** → menu **File** → **Import Notebook** → choose `kaggle/train_swarm_rl.ipynb`.
3. **Attach the code.** In the notebook's right-hand panel: **Input** → **Add Input** → search for your `swarm-rl-code` dataset → **Add**.
4. **Switch the GPU on.** Right-hand panel → **Session options** → **Accelerator** → **GPU P100** (or **GPU T4 x2**; the trainer uses one GPU). Internet can stay off.
5. **Choose the seed.** In the first code cell: `SEED = 1` (later runs: 2, 3). Leave `HOURS = 10.5`.
6. **Run it as a background job:** **Save Version** → **Save & Run All (Commit)** → **Save**. This runs on Kaggle's machines from top to bottom even if you close the browser or switch the laptop off. (Do not just press "Run all" in the editor: an interactive session stops after 20 minutes without activity.)
7. **Watch it (optional).** Open the notebook's page → the running version → **Logs**. Every few minutes a line like `step 123,456,789 fps 95,000 ret 1.23 success 0.41 crash_free 0.55 ...` appears, and every 30 minutes an `[eval] {...}` line with the scores on the validation courses.

What the notebook does, and how long it takes:

Cell	What happens	Time
1	prints the GPU name (must say <code>CUDA True</code>), unpacks the code	seconds
2	builds the courses: 8,000 training, 90 validation (to pick the best policy), 90 test (for the final report only)	~10 min
3	trains on the GPU for <code>HOURS = 10.5</code> h; checkpoint every 20 min; scores the 90 validation courses every 30 min and keeps the best policy	10.5 h
4	final fair test on the CPU with exactly the laptop's method: 90 test courses (<code>scripts/rl_eval.py</code>) and the real Islamabad map route (<code>scripts/rl_eval_route.py</code>)	~20–30 min
5	packs everything into <code>/kaggle/working/results.zip</code>	seconds

The code cells are numbered from the top (the text cell above them does not count); each one starts with a comment such as `# Cell 3: training on the GPU`.

Total about 11 hours, inside Kaggle's 12-hour limit. The trainer stops itself when `HOURS` are used up and saves everything first, so nothing is lost at the limit.

Fairness rule (important). The best policy is chosen on the 90 *validation* courses (`val.npz`, seeds 1,000,000–1,000,029). The reported results use the 90 *test* courses (`heldout.npz`, seeds 1,000,100–1,000,129) that neither training nor selection ever saw — the same courses as the laptop comparison, so the numbers are directly comparable. Never pass `heldout.npz` to the trainer. (This was also fixed on 27 September 2026; before that the kit would have chosen the policy on the test courses.)

6. When the run is finished: what to download

On the notebook page open the finished version → **Output** tab → download `results.zip` (a few MB). Unzip it, for example to `~/Downloads/results/`. Inside (for seed 1):

In results.zip	What it is
<code>run_seed1/best_policy.npz</code>	the new brain — the policy with the best validation score
<code>run_seed1/policy.npz</code>	the policy at the very end of training (usually not the one to use)
<code>run_seed1/progress.csv</code>	training log: steps, speed, return, success, crashes, losses (one row per update)
<code>run_seed1/eval.jsonl</code>	validation scores every 30 minutes, per density level
<code>run_seed1/ckpt.pt</code>	full checkpoint (network, optimiser, step count) to continue training later
<code>run_seed1/status.json</code> , <code>session_*.json</code>	how far it got, and the exact settings and GPU of the session
<code>eval_seed1/summary.md</code> , <code>summary.json</code> , <code>episodes.jsonl</code> , <code>comparison.png</code>	the fair test : all five methods on the 90 test courses, same table as the laptop's
<code>route_eval_seed1/summary.md</code> , <code>episodes.jsonl</code>	the real-map test: F-9 Park → Faisal Mosque, 932 buildings, 5 runs, with and without the leader killed

7. Where to paste each file (on the laptop)

From results.zip	Paste it here in the repository	Why
the whole <code>run_seed1/</code> folder	<code>reports/logs/rl/kaggle_seed1/</code>	the training record (evidence)
the whole <code>eval_seed1/</code> folder	<code>reports/logs/rl/eval_kaggle_seed1/</code>	the fair comparison
the whole <code>route_eval_seed1/</code> folder	<code>reports/logs/rl/route_eval_kaggle_seed1/</code>	the real-map result
<code>run_seed1/best_policy.npz</code>	<code>models/avoid_policy.npz</code> — only if it is better (section 8); first rename the old one to <code>models/avoid_policy_run1.npz</code>	the app, the simulators and the agent all read this file

The same as commands, from the repository root:

```
mkdir -p reports/logs/rl/kaggle_seed1
cp -r ~/Downloads/results/run_seed1/. reports/logs/rl/kaggle_seed1/
cp -r ~/Downloads/results/eval_seed1 reports/logs/rl/eval_kaggle_seed1
cp -r ~/Downloads/results/route_eval_seed1 reports/logs/rl/route_eval_kaggle_seed1
```

Only after the check in section 8 says it is better:

```
git mv models/avoid_policy.npz models/avoid_policy_run1.npz
cp reports/logs/rl/kaggle_seed1/best_policy.npz models/avoid_policy.npz
```

For seeds 2 and 3 use `kaggle_seed2`, `eval_kaggle_seed2`, ... in the same way.

8. Is the new policy better? (decide before replacing)

Open `reports/logs/rl/eval_kaggle_seed1/summary.md` next to the laptop's `reports/logs/rl/eval/summary.md` and compare the **RL + brake** row (laptop: 26/30, 18/30, 5/30).

Use the new policy if all of these hold:

1. RL + brake completes at least as many test courses at every density, and more in total;
2. its crashes per mission are not higher than the laptop policy's at any density;
3. in `route_eval_kaggle_seed1/summary.md`, RL + brake still has 0 drones that hit something (laptop: 0 in 5 runs, with and without the leader killed).

If only some hold, keep the laptop policy and write the result down anyway (a negative result is still a result). With several seeds, pick the seed whose policy is best on these rules and report all seeds, not only the best one.

9. After replacing: check on the laptop and update the documents

```
# 1. the same test on the laptop (should give the same numbers as eval_seed1; takes ~9 min)
PYTHONPATH=src python3 scripts/rl_eval.py --policy models/avoid_policy.npz --out
reports/logs/rl/eval_kaggle_check
PYTHONPATH=src python3 scripts/rl_eval_route.py --policy models/avoid_policy.npz --out
reports/logs/rl/route_eval_kaggle_check
# 2. learning curve of the Kaggle run
python3 scripts/plot_rl_training.py reports/logs/rl/kaggle_seed1 --out
reports/rl_training_kaggle_seed1.png
# 3. tests still pass
python3 -m pytest
```

Then write the new numbers into `docs/RESULTS.md` (the table under "Obstacle avoidance"), rebuild the documents and commit:

```
PYTHONPATH=src python3 scripts/make_brief.py
PYTHONPATH=src python3 scripts/make_handover_pdf.py
git add models reports docs && git commit -m "RL: Kaggle seed 1 policy, results"
```

10. What you get at the end

- A new `models/avoid_policy.npz`. The ground-control app uses it the next time it starts (avoidance options "RL policy" and "RL policy + brake"), and so do the fast simulator, the evaluation scripts and the drone agent — no code change.
- A fair, reproducible comparison against the classical avoider on the same 90 test courses, plus the real-map result, with all raw logs in `reports/logs/rl/`.
- Learning curves showing whether more training still helps.
- With seeds 2 and 3: evidence that the result repeats (or does not).
- A checkpoint (`ckpt.pt`) to continue training later without starting again.

11. How to change things

To change	Where	Notes
seed	notebook cell 1, <code>SEED</code>	1, 2, 3 ...; each seed is a separate run
training time per session	notebook cell 1, <code>HOURS</code>	keep ≤ 10.5 so pools + evaluation fit in 12 h
number of parallel missions	cell 3, <code>'--envs', '1024'</code>	lower to 512 on "CUDA out of memory"
network size	cell 3, add <code>'--hidden', '256', '256'</code>	the app reads any number of layers; bigger is slower
learning rate	cell 3, add <code>'--lr', '1e-4'</code>	default 3e-4
total step cap (all sessions)	cell 3, add <code>'--steps', '5e9'</code>	default 3e9 agent steps; the time limit usually stops first
how often it checkpoints / scores	<code>'--checkpoint-min' (20), '--eval-min' (30)</code>	minutes
exploration noise at the start	<code>'--log-std-init' (-1.0)</code>	lower = calmer start
reward	<code>src/swarm_tools/obstacle_sim.py</code> and <code>src/swarm_tools/torch_sim.py</code>	the two must stay identical: change both, then run <code>PYTHONPATH=src .venv/bin/python -m pytest tests/test_torch_sim.py</code> on the laptop
obstacle density of the training courses	<code>SimCfg.building_density</code> , <code>SimCfg.tree_density</code> in <code>obstacle_sim.py</code>	the test levels (<code>LEVELS</code> in <code>rl_eval.py</code> / <code>build_pools.py</code>) stay fixed so results remain comparable
inputs of the network	<code>avoidance.py</code> <code>observation()</code> and the batched version in <code>torch_sim.py</code>	changes <code>OBS_DIM</code> : old policies stop loading (the loader says so); retrain from scratch
number of drones	<code>'--drones' (10)</code>	also pass <code>--drones</code> to the evaluation scripts

Every code change needs a new zip on Kaggle: run `python3 scripts/make_kaggle_bundle.py`, open your `swarm-rl-code` dataset on Kaggle → **New Version** → upload the new zip; in the notebook's **Input** panel make sure the latest dataset version is attached; then Save & Run All again.

Continue a seed that stopped at the time limit (the log ends with `[budget] ... saved` and `status.json` says `"done": false`): in a new version of the notebook, **Add Input** → the previous version's output (search your notebook's name), and insert this cell after cell 2:

```
import glob, shutil
prev = glob.glob('/kaggle/input/**/run_seed1', recursive=True)[0]
shutil.copytree(prev, '/kaggle/working/run_seed1', dirs_exist_ok=True)
```

The trainer finds `ckpt.pt` and continues from the saved step, keeping the best score so far.

12. The whole procedure on one page

What gets trained? Only the followers' obstacle-avoidance policy, a small neural network. Every 0.1 s it reads 47 numbers (where the formation wants the drone to go, its own speed, its distance from its place in the V, the distance to walls and trees in 24 directions, the three nearest drones) and returns 2 numbers: how much to change its velocity. The leader election, the formation rules and the autopilot are not trained; they are rules and stay as they are.

Which problem does it solve? The laptop training ran for 38 minutes and was still improving when it stopped. In dense clutter only 5 of 30 missions succeed today. About 10 hours on Kaggle's GPU should reduce crashes; by how much can only be measured. Seeds 2 and 3 show whether the result repeats.

Which files to take? Only two: 1. `kaggle/swarm-rl-code.zip`: first run `python3 scripts/make_kaggle_bundle.py` on the laptop, which builds a fresh zip (your map keys are never included). 2. `kaggle/train_swarm_rl.ipynb`: the notebook.

How to run it on Kaggle: 1. `kaggle.com` → Create → New Dataset → upload the zip → name `swarm-rl-code` → Private → Create. 2. Create → New Notebook → File → Import Notebook → choose `train_swarm_rl.ipynb`. 3. Right-hand panel: Input → Add Input → add your `swarm-rl-code` dataset. 4. Session options → Accelerator → GPU P100 (or T4 x2). Internet can stay off. 5. Keep `SEED = 1` in the first cell (next time 2, then 3). 6. Save Version → **Save & Run All (Commit)** → Save. Kaggle keeps running even if you switch the laptop off. It takes about 11 hours.

When training has finished: 1. On the notebook's page open that version's **Output** tab → download `results.zip` → unzip it into `~/Downloads/results/`. 2. Paste these folders into the project (or copy the commands from section 7): - `run_seed1/` → `reports/logs/rl/kaggle_seed1/` - `eval_seed1/` → `reports/logs/rl/eval_kaggle_seed1/` - `route_eval_seed1/` → `reports/logs/rl/route_eval_kaggle_seed1/` 3. Compare the "RL + brake" line in `eval_kaggle_seed1/summary.md` with the old one (26/30, 18/30, 5/30). Only if it is equal or better at every density, crashes are not higher, and the Islamabad route still has 0 hits, switch to the new policy: rename `models/avoid_policy.npz` to `avoid_policy_run1.npz` and copy `run_seed1/best_policy.npz` to `models/avoid_policy.npz`. 4. Re-check on the laptop with the commands in section 9, write the new numbers into `docs/RESULTS.md`, rebuild the PDF, commit.

What you get at the end: a new `models/avoid_policy.npz`. The app's "RL policy" and "RL policy + brake" options, the simulators and the drone agent use it without any code change. Plus a fair comparison (the same 90 test courses as on the laptop), the real-map result, and the training curves.

To change something: seed and hours in the notebook's first code cell; everything else in the third cell (table in section 11). If you change code, build a new zip and upload it as a "New Version" of the Kaggle dataset. To change the reward, change `obstacle_sim.py` and `torch_sim.py` in the same way and run `tests/test_torch_sim.py` to check they still match.

13. When something goes wrong

What you see	Cause and fix
cell 1 prints <code>CUDA False</code>	the GPU is not on: Session options → Accelerator → GPU (phone verification may be needed)
cell 1: <code>IndexError: list index out of range</code>	the <code>swarm-rl-code</code> dataset is not attached: Input → Add Input
<code>CUDA out of memory</code>	use <code>'--envs', '512'</code> in cell 3

What you see	Cause and fix
the version stops at 12 h without <code>results.zip</code>	<code>HOURS</code> too high or the evaluation was slow: set <code>HOURS = 10</code> , and continue the seed (section 11)
the version shows "Error"	open Logs , read the last lines; fix, make a new zip if the code changed, Save & Run All again
<code>policy expects N inputs, this code builds 47</code> when loading a policy	the policy was trained with a different input layout; retrain, or use the matching code version
the laptop check gives different numbers than <code>eval_seed1</code>	should not happen (same scripts, same courses); check that the laptop code is the same version as the zip
the app does not offer the RL options	<code>models/avoid_policy.npz</code> is missing
training <code>success</code> in the log stays at 0 for hours	normal in the first hour; if it never rises, compare the <code>[eval]</code> lines with <code>reports/logs/rl/run1/eval.jsonl</code> and check the reward change you made